Basic Git

Samed Chodankar

Abstract—This report provides a fundamental overview of Git, a distributed version control system. It outlines the core concepts of version control, local repository management, remote synchronisation, and branch integration techniques.

I. Introduction

Git is a distributed version control system that allows developers to track changes in their source code over time. It is designed to coordinate work among programmers, but it can be used to track changes in any set of files. Unlike centralised version control systems, Git provides every user with a complete local copy of the repository. This structure ensures that operations are remarkably fast and do not strictly require an active internet connection to perform standard tasks.

II. Core Concepts

Before executing commands, it is essential to understand the underlying architecture of a Git repository.

- **Repository:** A directory that contains the project files and the complete history of all changes made to those files.
- **Commit:** A snapshot of the repository at a specific point in time. Commits form a chronological chain or tree, allowing users to revert to previous states if a mistake occurs.
- **HEAD:** A special pointer that indicates the currently checked-out commit or branch. It is effectively the "current location" of the user within the repository structure.

III. Local Operations

Local operations are performed entirely on the computer of the user without needing to communicate with a remote server.

- **Commit (git commit):** When modifications are made and saved, they are not automatically added to the version history. The commit command captures the current state of the tracked files and stores it as a new, permanent node in the Git history tree.

- **Branch (git branch):** A branch is simply a lightweight, movable pointer to a specific commit. Branching allows developers to diverge from the main line of development and continue working independently on a new feature or fix without affecting the primary codebase.
- **Checkout (git checkout):** This command is used to navigate between different branches or commits. Executing this command updates the files in the working directory to match the version stored in the specified branch, and it moves the HEAD pointer to that new location.

IV. Remote Operations

Remote operations allow developers to collaborate by sharing commits across different repositories, typically hosted on services like GitHub or GitLab.

- **Clone (git clone):** This command creates a complete local copy of a remote repository. It downloads all the data, branches, and version history, automatically setting up a link between the local copy and the remote source.
- **Fetch (git fetch):** Fetching contacts the remote repository and downloads any new commits or branches that do not currently exist in the local repository. However, it does not alter the local working directory or merge the new data; it merely updates the remote-tracking branches.
- **Pull (git pull):** This command is a direct combination of two operations. It first fetches the data from the remote repository and then automatically merges it into the currently active local branch. It is used to synchronise the local repository with upstream updates quickly.
- **Push (git push):** Pushing is the exact opposite of fetching. It uploads the local branch commits to the remote repository, allowing other collaborators to access the new changes.

V. Integration Operations

When working with multiple branches, it is eventually necessary to integrate changes from one branch into another.

- **Merge (git merge):** Merging takes the contents of a source branch and integrates them into the target branch. It creates a new "merge commit" that ties the histories of both branches together. This preserves the complete, factual history of exactly how the work was developed.
- **Rebase (git rebase):** Rebasing is a structural alternative to merging. It takes the sequence of commits from the current branch and reapplies them sequentially on top of another branch. This results in a much

cleaner, linear project history, as it avoids creating extra merge commits.

- **Cherry-pick (git cherry-pick):** This command allows a user to select specific, individual commits from any location in the repository and apply a copy of them directly below the current HEAD location. It is highly useful when only a specific bug fix or isolated feature is needed from another branch, rather than merging the entire branch.

VI. Conclusion

Git provides a framework for managing code versions and facilitating structured collaboration. By mastering local commands like commit and checkout, understanding remote synchronisation through fetch, pull, and push, and learning integration strategies like merge and rebase, a user can efficiently navigate standard development workflows. These core concepts form the necessary foundation for all modern software engineering collaboration.